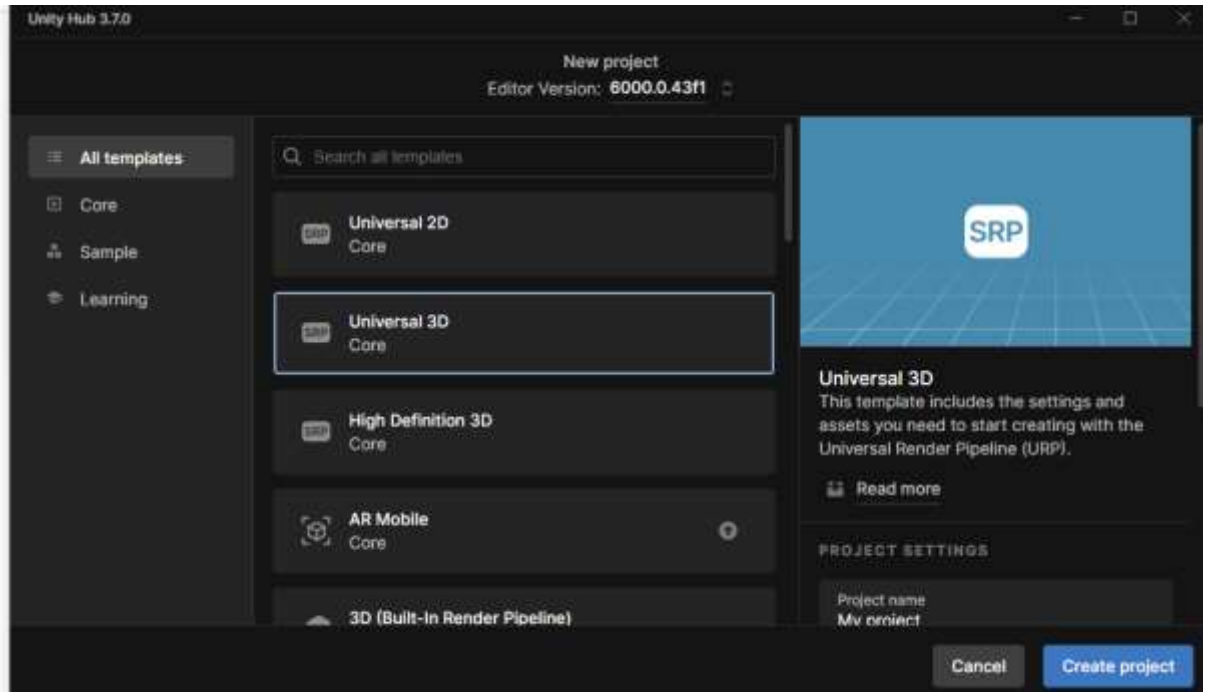
UNITY 6 AR Setup

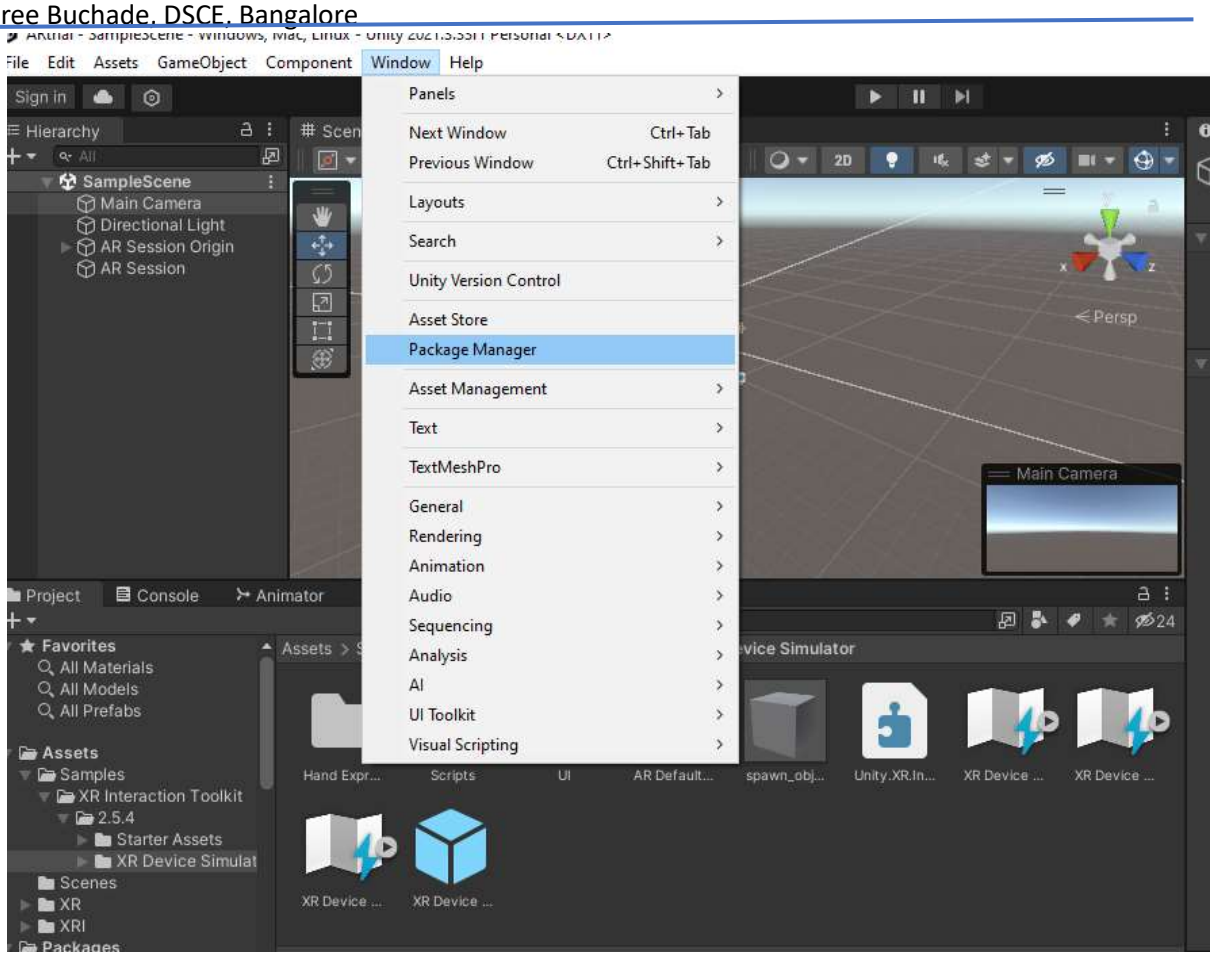
1. Create Project 3D

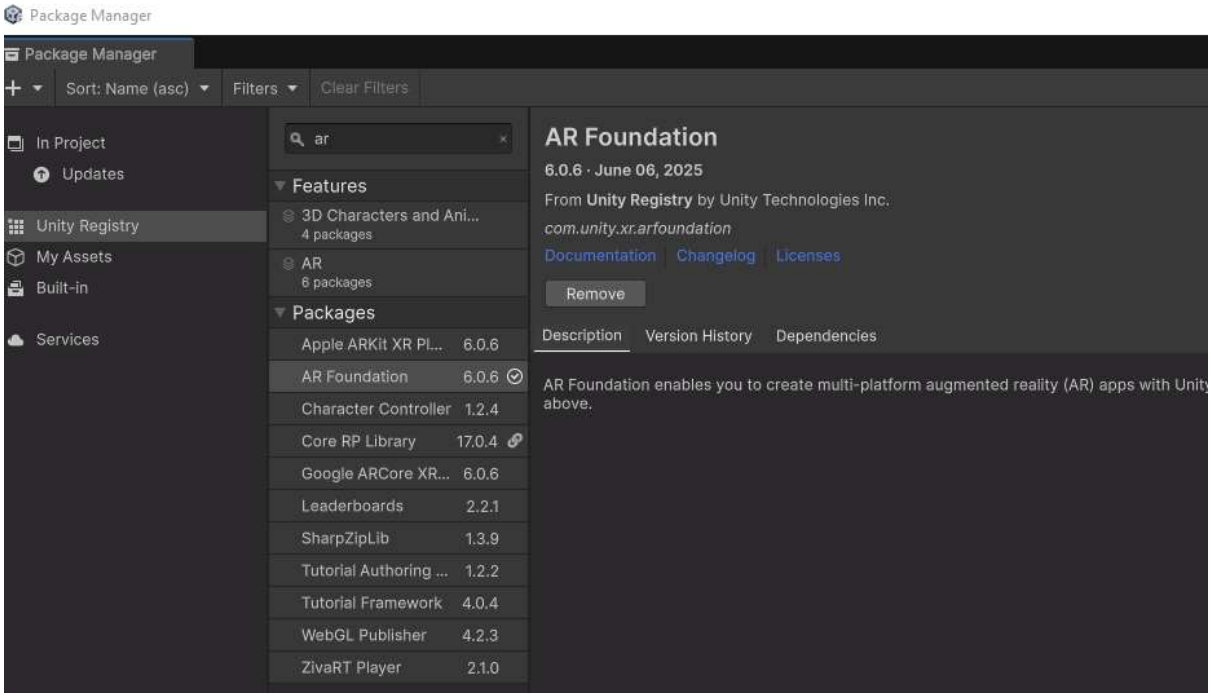


Note: It will work even for 3D built-in render pipeline if you want less processing and medium quality application.

2. Add packages for AR

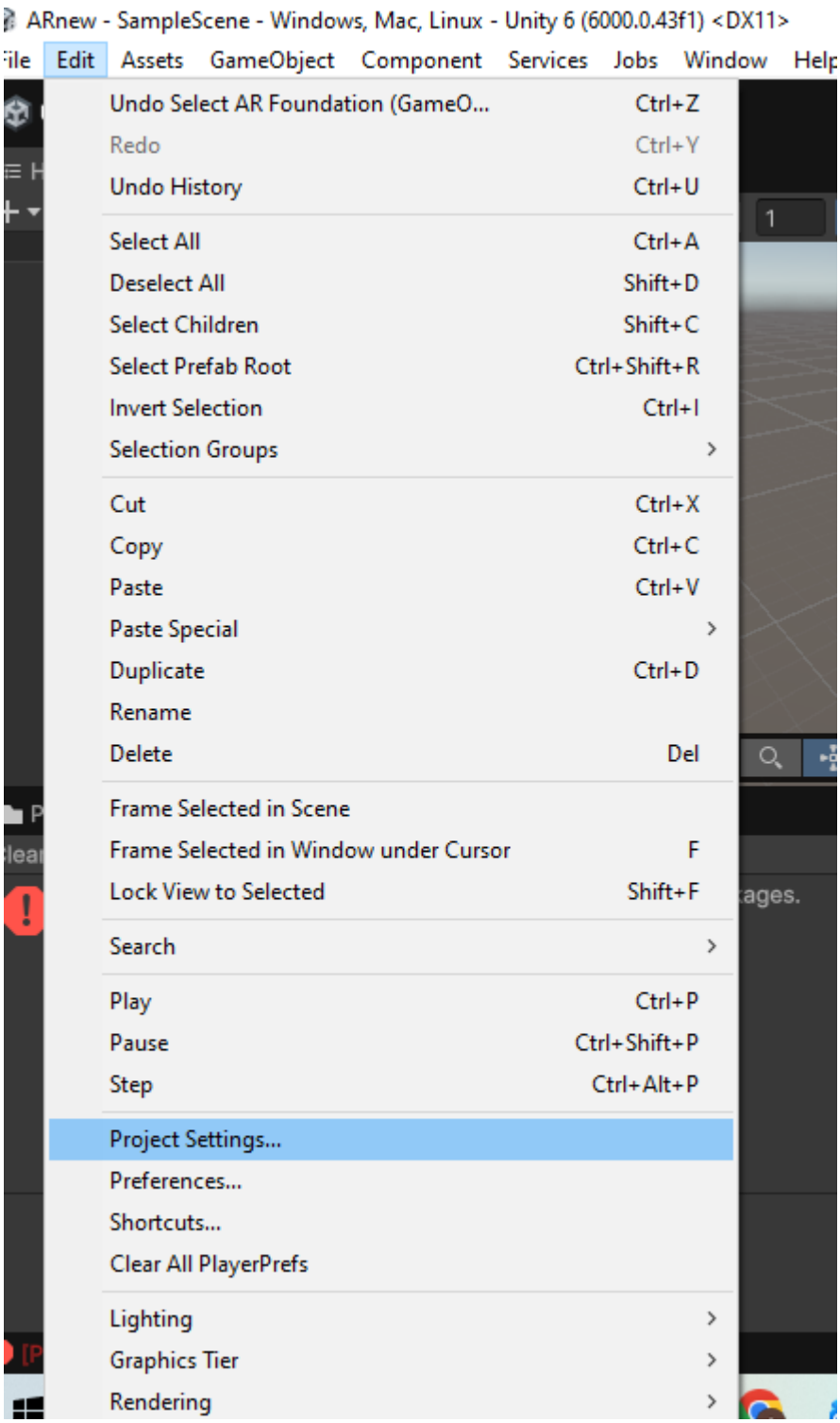
- Windows -> Package Manager -> Unity registry Add AR Foundation and google ARcore for Android and Arkit for Apple



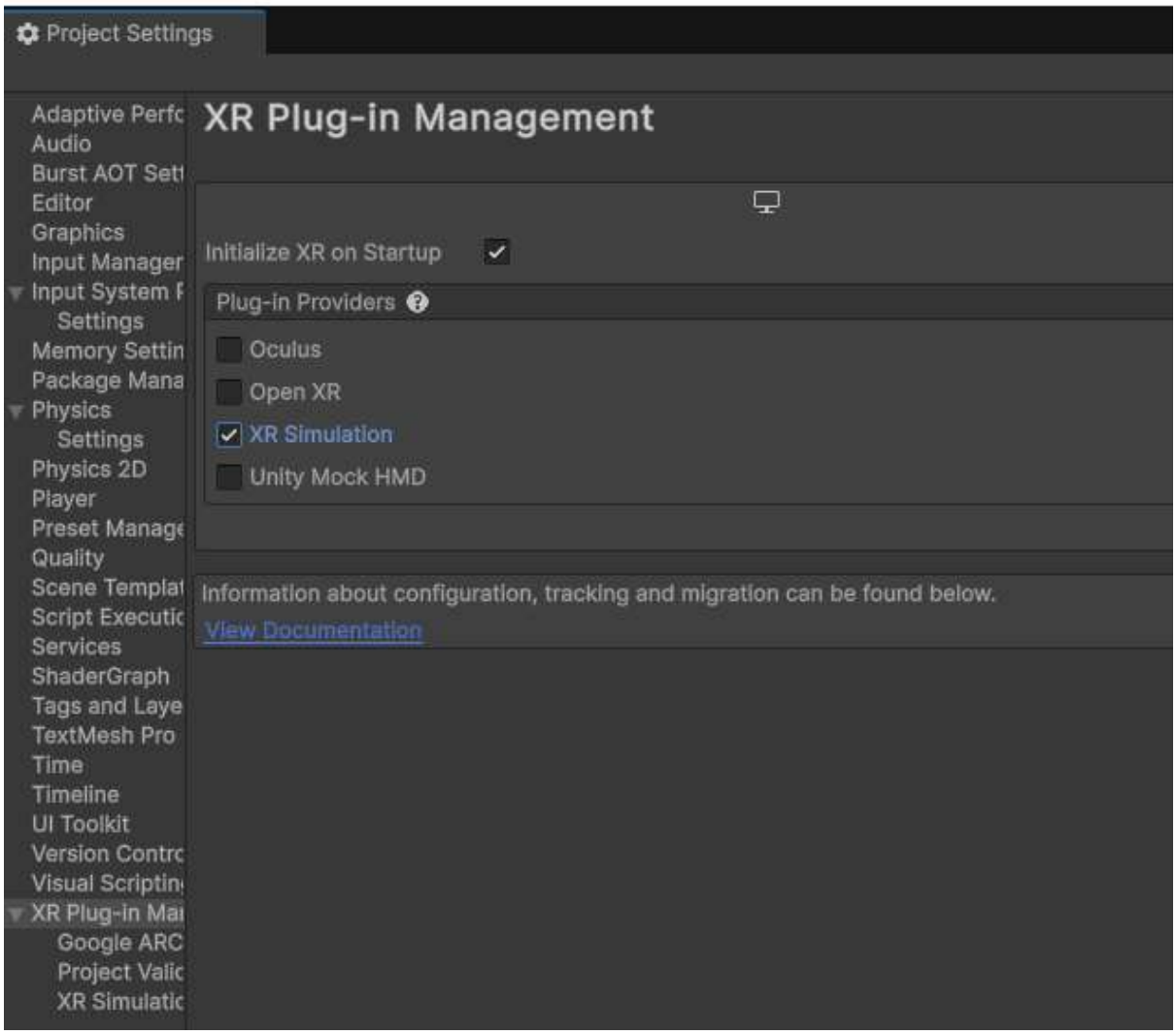


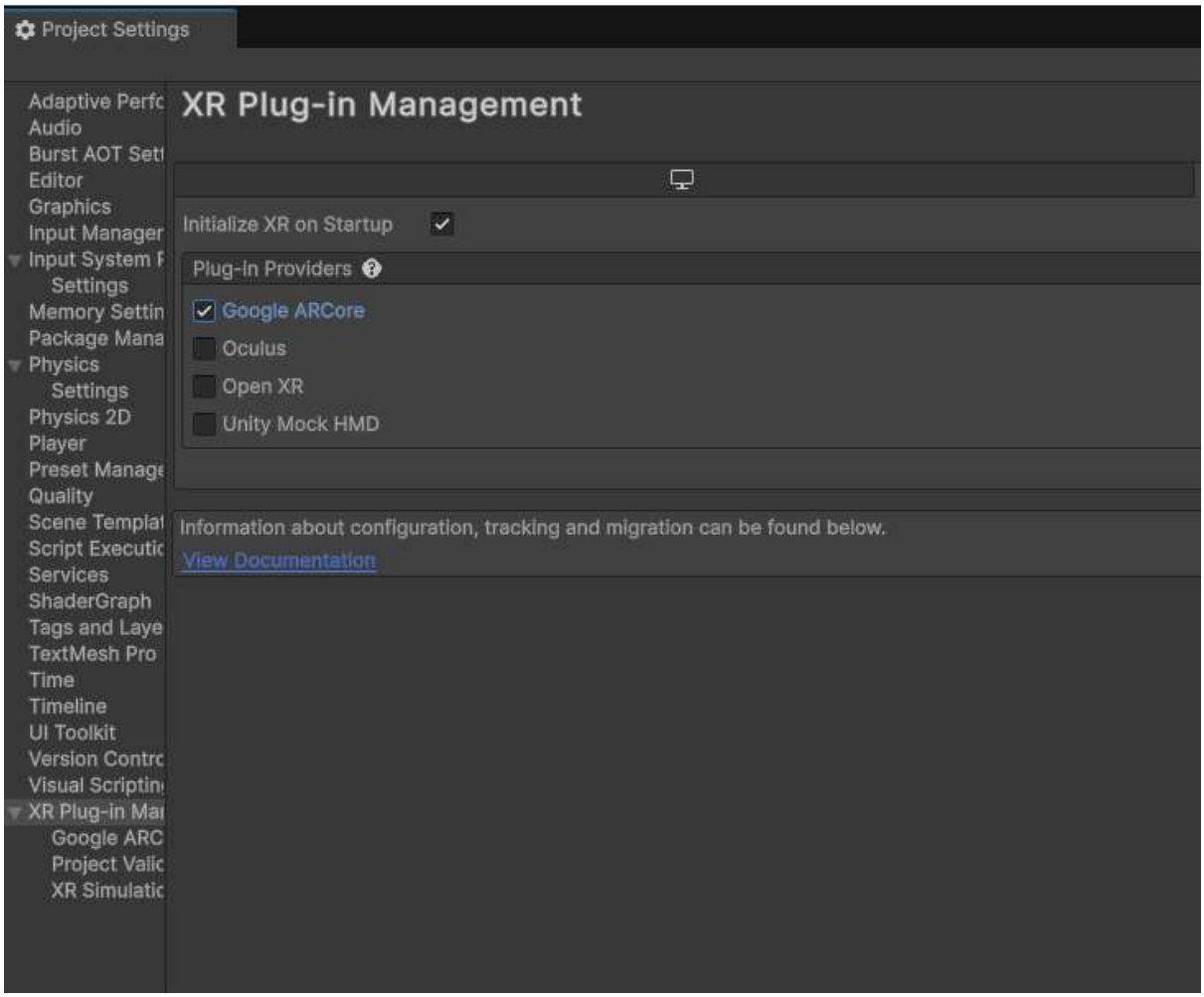
- Install both

3. Edit -> Project settings -> open Tab PC and android one by one and enable XR simulation and AR core for Android and if Apple ARkit

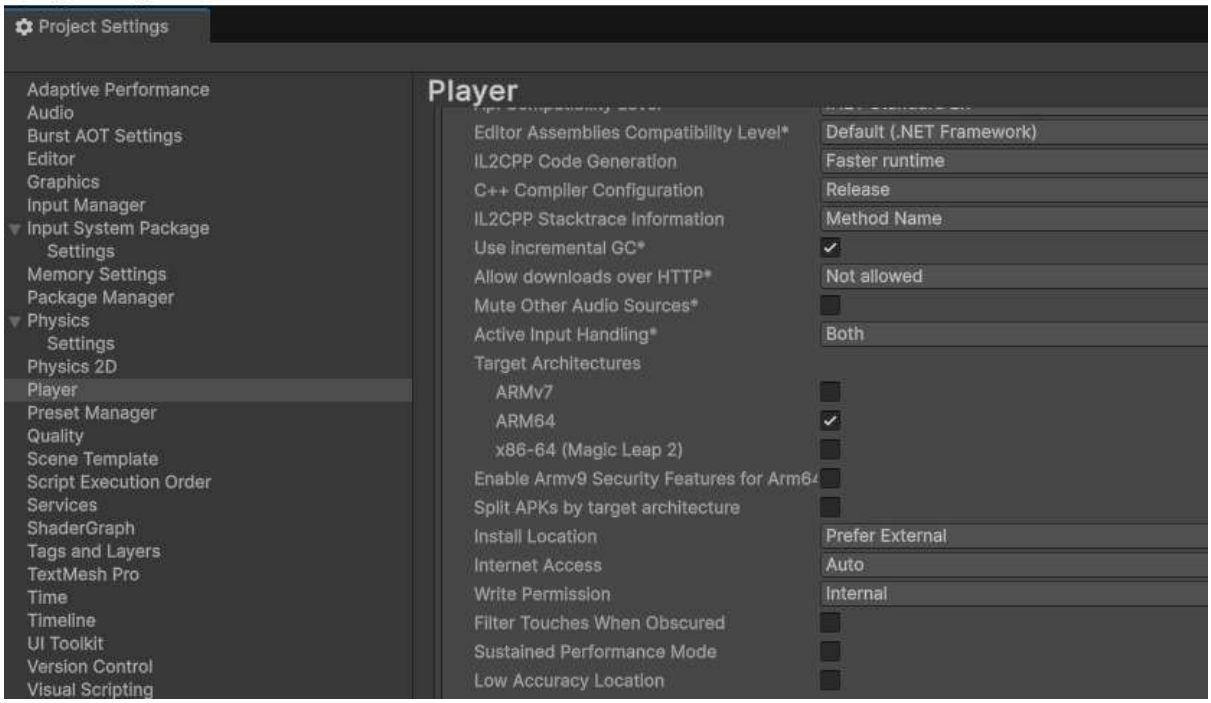


Project Settings

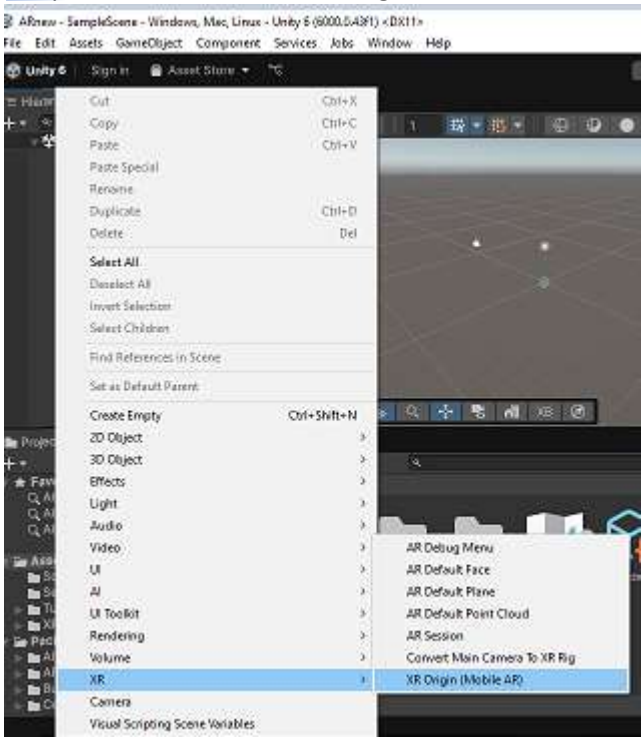




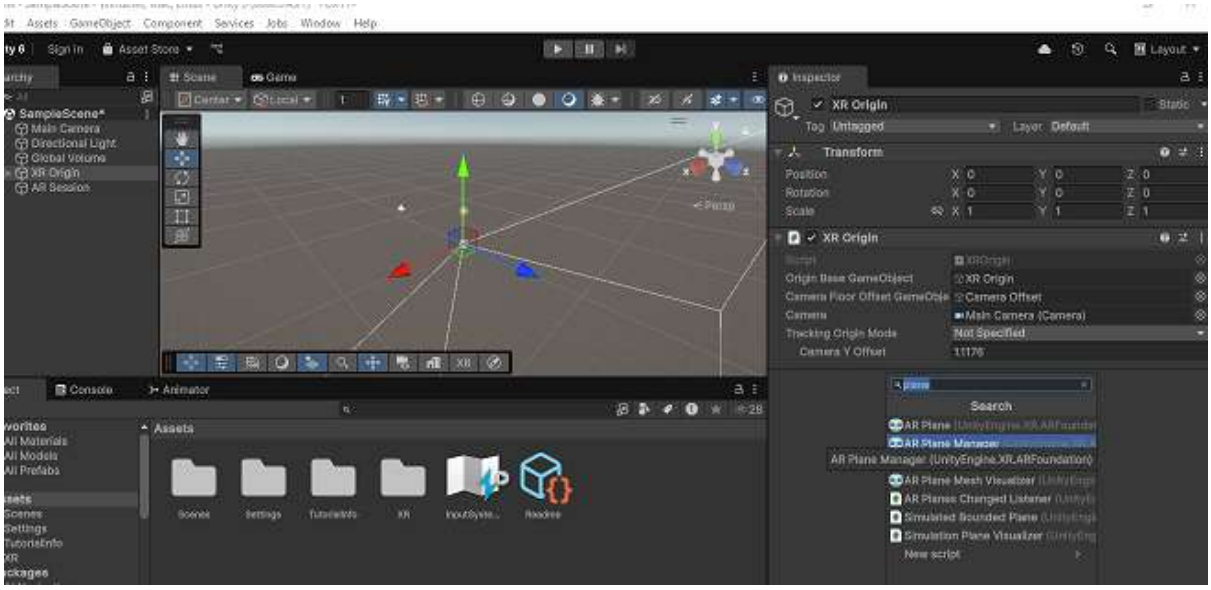
- Select project validation nad click Fix ALL
- Select player ->other setting -> see Input handling select to both



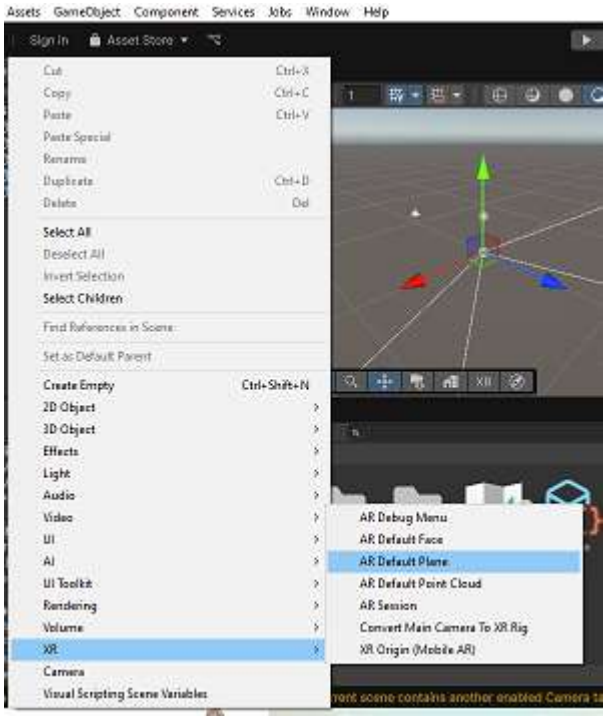
4. In hierarchy window Right Click -> Add XROgrin and ARsession



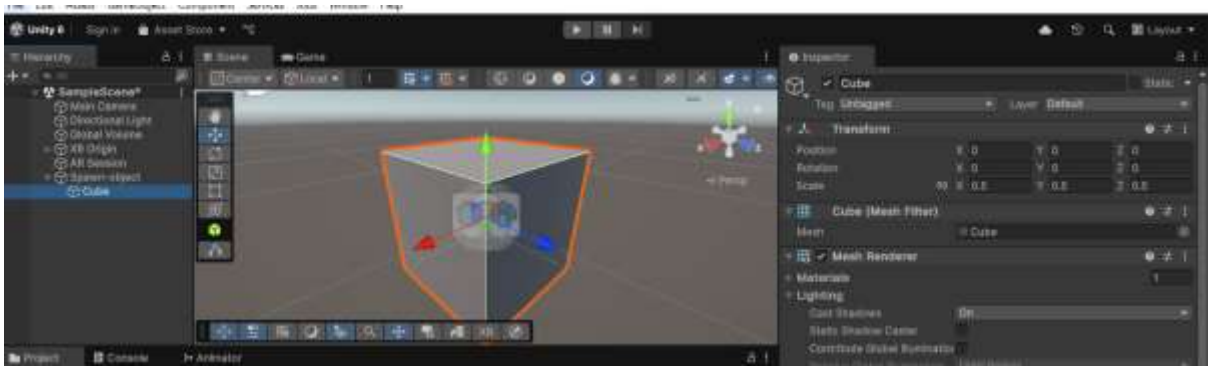
- Select XROrgin -> click Add component ->Add plane manager and raycast manager



- Right click in Hierarchy window selct XR->Ar default plane -> Drag it in project window it will be prefab and delete from hierarchy add it it plane manager

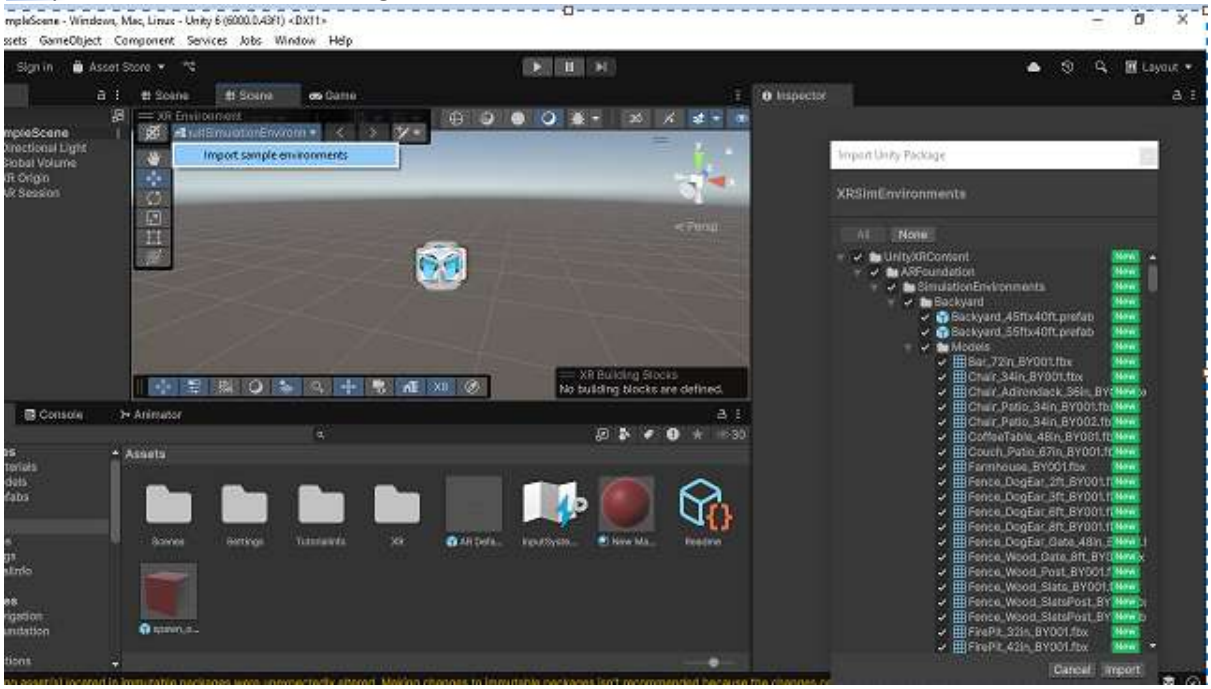


- Right click create empty game object I named it spawn_object
    - Select spawn_object and create 3D cube and sacle it down

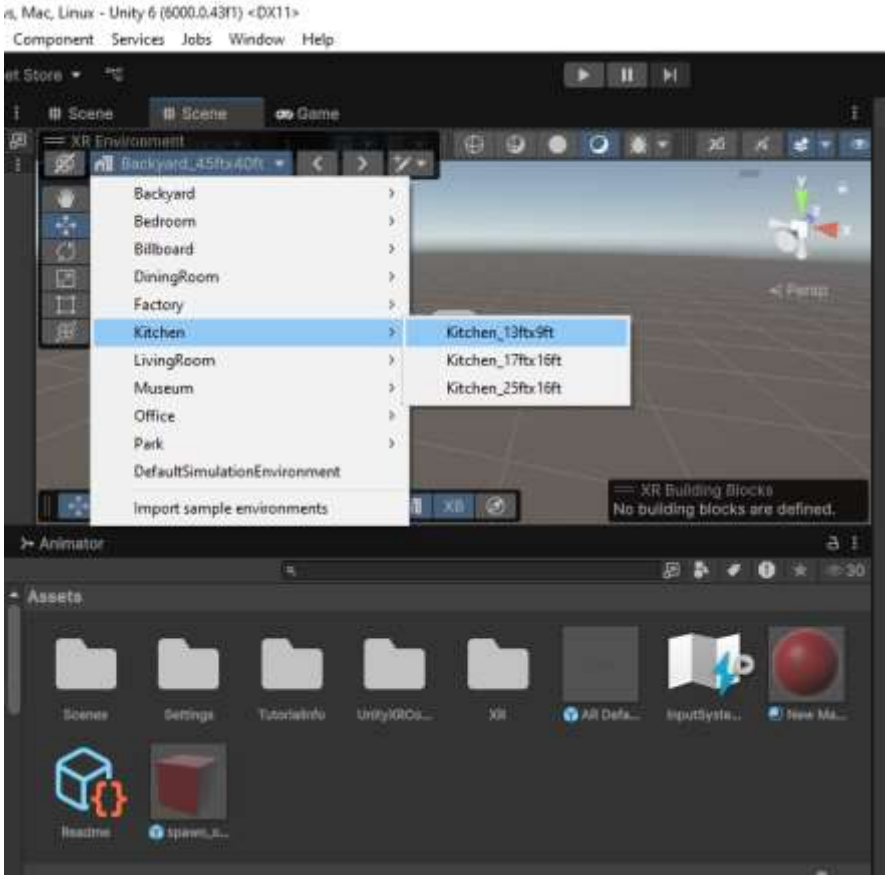


5. Add simulator

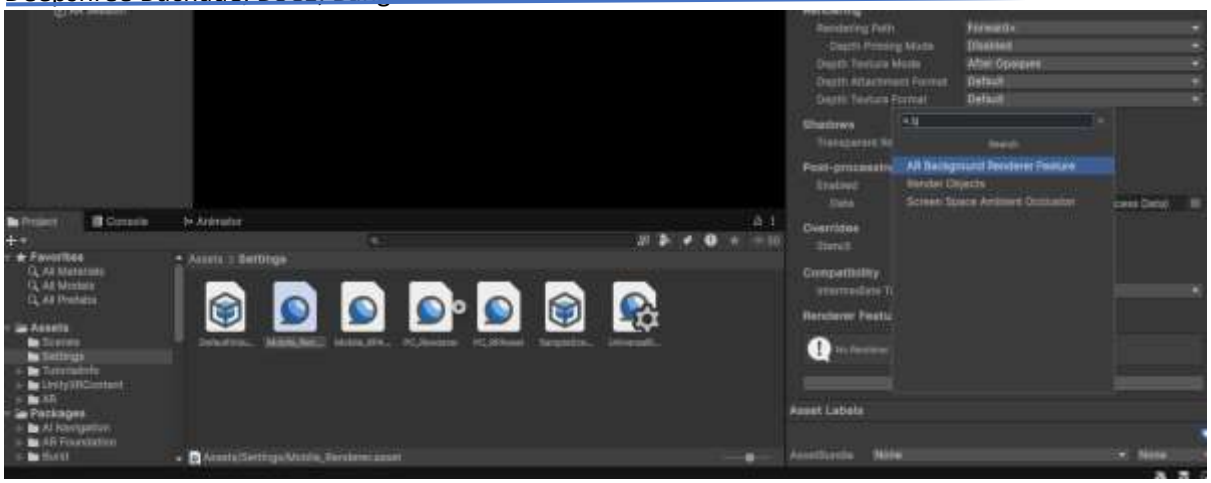
- Click the Import environment ->click import



- You can select sample

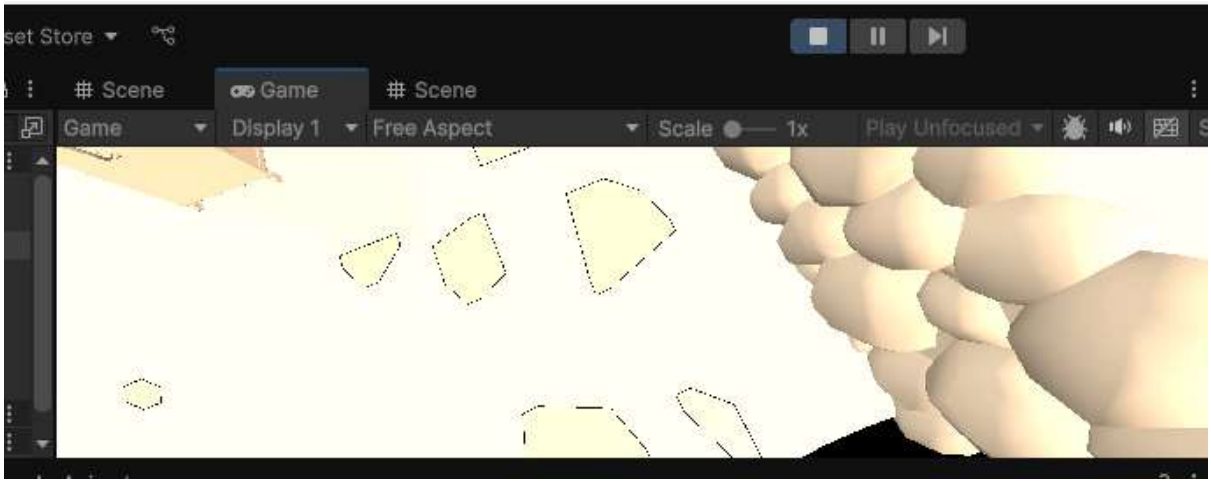


- Project window -> setting -> Modbile render add feature



- Same for Pc Renderer

Run and use Alt and Right mouse Button to see plane is detected



6. Create Mono behaviour script and attached to XR origin

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class ARPlaceCube : MonoBehaviour
{
    [SerializeField] private ARRaycastManager raycastManager;
    [SerializeField] private GameObject spawnPrefab;
    [SerializeField] private float scaleStep = 0.1f;
    [SerializeField] private float minScale = 0.1f;
    [SerializeField] private float maxScale = 3f;
    [SerializeField] private float rotateStep = 15f;

    private bool isPlacing = false;
    private GameObject spawnedObject = null;

    void Update()
```

```
    {
        if (!raycastManager) return;

        if (spawnedObject == null && !isPlacing)
        {
            if (Input.touchCount > 0 && Input.GetTouch(0).phase == TouchPhase.Began)
            {
                isPlacing = true;
                PlaceObject(Input.GetTouch(0).position);
            }
            else if (Input.GetMouseButtonDown(0))
            {
                isPlacing = true;
                PlaceObject(Input.mousePosition);
            }
        }

        if (spawnedObject != null)
        {
            if (Input.GetKeyDown(KeyCode.Equals) || Input.GetKeyDown(KeyCode.KeypadPlus))
                ChangeScale(scaleStep);

            if (Input.GetKeyDown(KeyCode.Minus) || Input.GetKeyDown(KeyCode.KeypadMinus))
                ChangeScale(-scaleStep);

            if (Input.GetKeyDown(KeyCode.RightArrow))
                spawnedObject.transform.Rotate(Vector3.up, -rotateStep, Space.World);

            if (Input.GetKeyDown(KeyCode.LeftArrow))
                spawnedObject.transform.Rotate(Vector3.up, rotateStep, Space.World);

            if (Input.touchCount == 2)
            {
                Touch t0 = Input.GetTouch(0);
                Touch t1 = Input.GetTouch(1);
                float prevDist = Vector2.Distance(t0.position - t0.deltaPosition, t1.position -
t1.deltaPosition);
                float currDist = Vector2.Distance(t0.position, t1.position);
                ChangeScale((currDist - prevDist) * 0.001f);
            }

            if (Input.touchCount == 1 && Input.GetTouch(0).phase == TouchPhase.Moved)
                spawnedObject.transform.Rotate(Vector3.up, -Input.GetTouch(0).deltaPosition.x * 0.1f,
Space.World);
        }
    }

    void PlaceObject(Vector2 screenPosition)
    {
        var rayHits = new List<ARRaycastHit>();
        raycastManager.Raycast(screenPosition, rayHits, TrackableType.AllTypes);

        if (rayHits.Count > 0)
        {
```

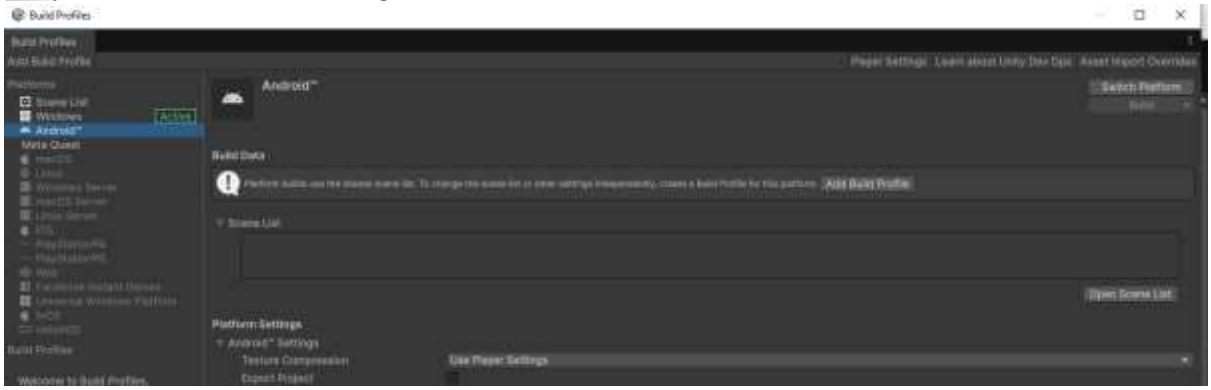
```
            GameObject prefab = spawnPrefab != null ? spawnPrefab : raycastManager.raycastPrefab;
            spawnedObject = Instantiate(prefab, rayHits[0].pose.position, rayHits[0].pose.rotation);
        }

        StartCoroutine(ResetPlacing());
    }

    void ChangeScale(float amount)
    {
        float newScale = Mathf.Clamp(spawnedObject.transform.localScale.x + amount, minScale,
maxScale);
        spawnedObject.transform.localScale = Vector3.one * newScale;
    }

    IEnumerator ResetPlacing()
    {
        yield return new WaitForSeconds(0.25f);
        isPlacing = false;
    }

    public void DeleteObject()
    {
        if (spawnedObject != null)
        {
            Destroy(spawnedObject);
            spawnedObject = null;
        }
    }
}
```

//drag raycast manager as refrence if not fund and run see in simulator

7. Run and see in simulator plane detection and click so one time object pop up

8. Create APk

- File Build Profile click android and switch platform



Click Build

VR Simulator

Step 1: Install Required Packages

Go to **Window → Package Manager → Unity Registry** and install:

1. **XR Plugin Management**
2. **XR Interaction Toolkit** (enable preview packages if needed)
    - After installing, import the **Starter Assets** sample
    - Import the **XR Device Simulator** sample

Step 2: Configure XR Plugin

1. Go to **Edit → Project Settings → XR Plugin Management**
2. Click **Install XR Plugin Management** if prompted
3. On the **PC tab**, check ✅ **OpenXR**
4. Go to **OpenXR** settings below and add **Interaction Profile**:
    - Add **Oculus Touch Controller Profile** (or any)

Step 3: Set Up the Scene

Replace Camera with XR Rig:

1. **Delete** the default Main Camera from the scene
2. In **Project window**, go to: Samples → XR Interaction Toolkit → [version] → Starter Assets
3. Drag **XR Origin (XR Rig)** prefab into the scene

Add Interaction Manager:

4. **Right-click Hierarchy → XR → Interaction Manager** (or check if it already exists in the rig)

Step 4: Add XR Device Simulator

1. In **Project window**, go to: Samples → XR Interaction Toolkit → [version] → XR Device Simulator
2. Drag **XR Device Simulator** prefab into the scene

Step 5: Create a Grabbable Object

1. **Right-click Hierarchy → 3D Object → Cube** (or any shape)
2. Position it in front of the rig (e.g., Z = 1, Y = 1)
3. With Cube selected, **Add Component**:
   - ✅ **Rigidbody** (if not already there)
   - ✅ **XR Grab Interactable**

Step 6: Add a Floor

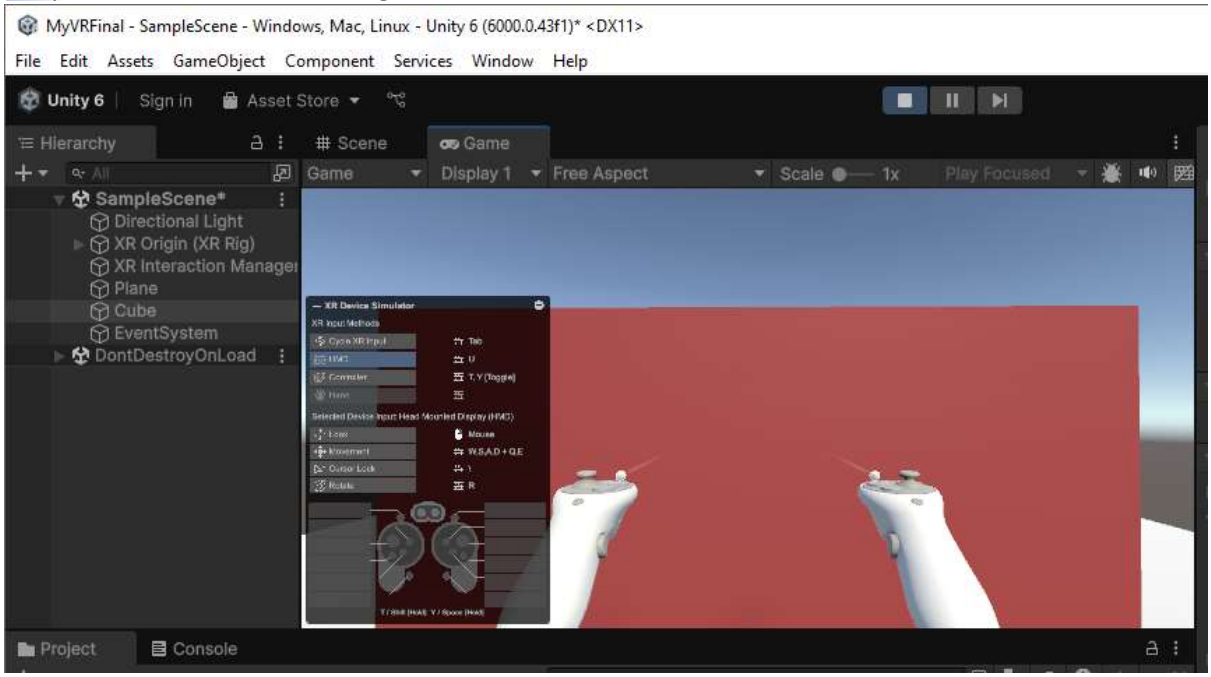
1. **Right-click Hierarchy → 3D Object → Plane**
2. Set position to (0, 0, 0) so objects land on it after dropping

---

Step 7: Press Play & Simulate

Use these controls with the **XR Device Simulator**:

Action	**Control**
Move in scene	W A S D
Look around	**Right-click** + drag mouse
Activate Left Hand	Hold Left Shift
Move Left Hand	Hold Left Shift + move mouse
Activate Right Hand	Hold Space
Move Right Hand	Hold Space + move mouse
Grip / Grab	Hold Left Shift or Space → press G
Trigger	Hold Left Shift or Space → press T
Drop	Release G



Use T or Tab to toggle between head set